

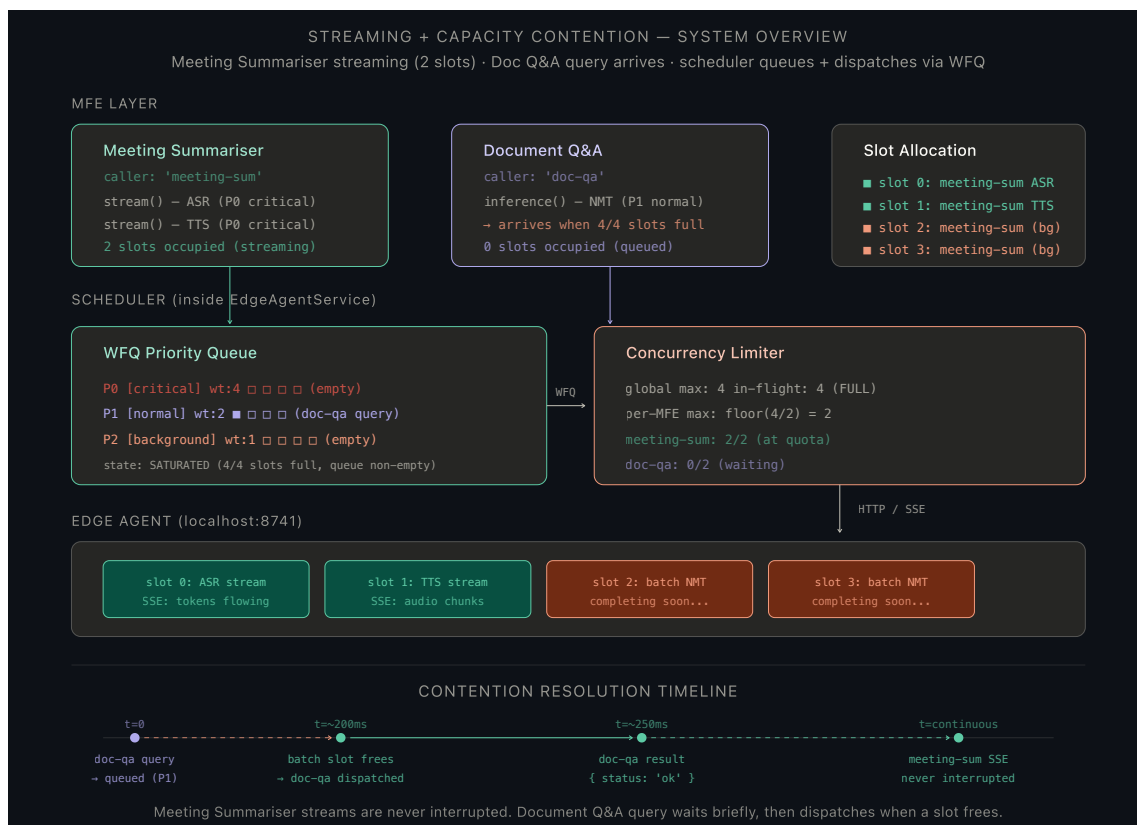
Part B §3 — Streaming + Capacity Contention Scenario

Scenario: The Meeting Summariser MFE is streaming tokens through SSE (live ASR transcription). While streaming is active, the user opens the Document Q&A MFE and submits a query. At this moment, the Edge Agent is already at capacity (4/4 active slots).

Overview

This document defines the exact UX contract between the scheduler and both MFEs during a streaming contention event. It answers: what does each MFE receive, when, and in what format — and what happens when things go wrong.

The contention scenario is the most revealing test of the scheduler design from Part B §2. It forces every mechanism — WFQ, per-MFE fairness, cancellation propagation, timeout, queue visibility — to work together under real load.



Assumptions

- Meeting Summariser holds 2 streaming slots** — one for ASR (speech-to-text) and one for TTS (text-to-speech). Both are P0 critical priority, long-running SSE streams.
- Two other slots are occupied** — by batch requests from either MFE (background prefetch, prior queries). The exact occupant doesn't matter; what matters is 4/4 slots are full.
- Document Q&A submits a P1 normal priority query** — a user-triggered translation or search. This is not background prefetch; the user is waiting.
- Per-MFE quota is floor(4/2) = 2** — both MFEs are active, so each gets at most 2 concurrent slots.

5. **The scheduler is in SATURATED state** — all 4 slots occupied, queue may or may not be empty before Doc Q&A's request arrives.
6. **SSE streaming holds a slot for the entire stream duration** — a streaming request does not release its slot between tokens. The slot is freed only when the stream sends `[DONE]`, times out, or is aborted.

1. System Architecture — Contention Layout

Slot allocation at moment of contention

Edge Agent Slots (4/4 occupied):

slot 0:	meeting-sum	ASR stream (P0)	— SSE tokens —>
slot 1:	meeting-sum	TTS stream (P0)	— SSE audio —>
slot 2:	[batch]	completing...	— HTTP 200 —>
slot 3:	[batch]	completing...	— HTTP 200 —>

Scheduler Queue:

P0 [critical]	□ □ □ □	(empty)
P1 [normal]	■ □ □ □	(doc-qa query — just arrived)
P2 [background]	□ □ □ □	(empty)

Per-MFE counters:

meeting-sum: 2/2 in-flight (at quota)

doc-qa: 0/2 in-flight (waiting)

Why Doc Q&A is queued, not rejected

The global limit (4) is full, but this is not an error condition — it's the expected steady state under load. The scheduler **enqueues** the request in the P1 lane and starts a 30-second timeout timer. Doc Q&A gets a queued notification, not an error.

Why Meeting Summariser's streams are not interrupted

Doc Q&A's request is P1 `normal`. Preemption only applies to P0 preempting P2. P1 cannot preempt P0 (that would break live transcription), and P0 cannot preempt P0 (streams are equally critical). The streams continue uninterrupted.

2. What Each MFE Receives

Meeting Summariser — uninterrupted stream

Meeting Summariser is completely oblivious to the contention. Its SSE streams continue delivering tokens with zero interruption, zero latency change, and zero notification of Doc Q&A's queueing.

```
// Meeting Summariser's perspective — nothing changes
for await (const chunk of stream({
```

```

    model: 'indic-asr-v1',
    input: audioChunk,
    priority: 'critical',
    caller: 'meeting-sum',
  ))) {
    // Tokens arrive at the same rate as before Doc Q&A opened
    appendTranscription(chunk.data.text);
    // chunk.meta.contention is NOT a field – MFEs don't see queue state
  }
}

```

Token delivery during contention:

Property	Value	Notes
Token latency	Unchanged (~5-15ms per token)	Scheduler forwards SSE events immediately, no buffering
Token ordering	Preserved	SSE is ordered by definition; scheduler does not reorder
Jitter	None introduced	Scheduler adds < 0.01ms overhead (async yield)
Backpressure	None applied	Streaming slots are not throttled by queue depth
Notification of contention	None	Meeting Summariser has no visibility into other MFEs' queue state

Document Q&A — queued, then dispatched

Document Q&A's `inference()` call returns a Promise that won't resolve immediately. The scheduler emits events on the `EdgeAgentService` event bus that the MFE can optionally subscribe to for queue position updates.

```

// Document Q&A's perspective
const result = await inference({
  model: 'indic-nmt-v2',
  input: 'Hello, how are you?',
  priority: 'normal',
  caller: 'doc-qa',
});

// Promise resolves after ~200-300ms (queue wait + inference time)
// result.meta.queued_ms tells the MFE how long it waited
if (result.status === 'ok') {
  showTranslation(result.data.text);
  // result.meta.queued_ms === 200 ← was queued for 200ms
}

```

Queue visibility via event subscription:

```
// Optional: subscribe to queue events for progress UI
const { status, events } = useEdgeAgent();

useEffect(() => {
  const unsub = events.on('scheduler:queued', (e) => {
    if (e.caller === 'doc-qa') {
      setQueuePosition(e.position); // "Queued – position 1"
    }
  });
  return unsub;
}, []);

useEffect(() => {
  const unsub = events.on('scheduler:dispatched', (e) => {
    if (e.caller === 'doc-qa') {
      setQueuePosition(null); // Clear queue indicator
      setLoading(true);      // Now processing...
    }
  });
  return unsub;
}, []);
```

3. When Updates Are Delivered

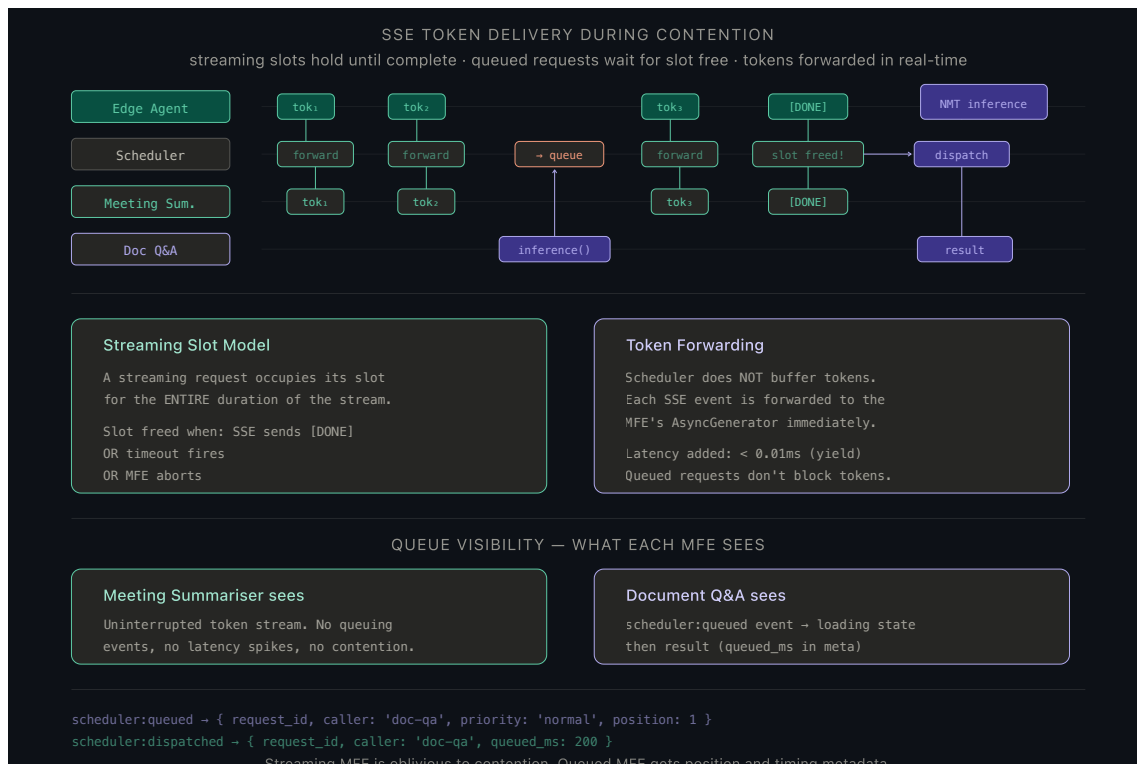
Timeline of the contention scenario

t=0ms	User opens Doc Q&A, submits query └ Scheduler: enqueue in P1 lane └ Scheduler emits: scheduler:queued { position: 1 } └ Doc Q&A: shows "Queued – position 1" └ Meeting Summariser: unaffected, SSE tokens continue
t=0-200ms	Batch request on slot 2 or 3 completes (200 OK) └ Scheduler: slot freed → dequeue loop fires └ WFQ selects P1 lane (only non-empty lane) └ Per-MFE check: doc-qa has 0/2 in-flight → OK └ Scheduler: dispatch doc-qa query to freed slot └ Scheduler emits: scheduler:dispatched { queued_ms: 200 } └ Doc Q&A: shows "Processing..."
t=200-250ms	Agent processes NMT inference └ Agent returns 200 OK with translation └ Scheduler: resolves Doc Q&A's Promise └ Doc Q&A: shows translated text
t=250ms+	Slot freed → dequeue loop fires → queue empty → IDLE or DISPATCHING

Meeting Summariser: still streaming, still unaffected

Update delivery guarantees

Event	Delivery timing	Guarantee
<code>scheduler:queued</code>	Synchronous with <code>inference()</code> call	Always emitted when request queued (not when dispatched immediately)
<code>scheduler:dispatched</code>	When slot is acquired and HTTP request is sent	Always emitted before any response data
SSE tokens	Real-time, forwarded within < 0.01ms of receipt	Never buffered, never delayed by queue activity
Promise resolution	After full HTTP response (or SSE <code>[DONE]</code>)	Exactly once per request
Timeout error	30s after <code>inference()</code> call (includes queue time)	Deterministic — covers both queue wait and inference



4. Event/State Formats

Scheduler events emitted during contention

```

// 1. When Doc Q&A's request is queued (t=0ms)
{
  type: 'scheduler:queued',
  request_id: 'req_a7f3',
  caller: 'doc-qa',
  priority: 'normal',
  position: 1, // 1-indexed position in the effective queue
  estimated_wait_ms: 200, // best-effort estimate based on current
throughput
  timestamp: '2025-01-15T10:23:45.000Z'
}

// 2. When Doc Q&A's request is dispatched (t=200ms)
{
  type: 'scheduler:dispatched',
  request_id: 'req_a7f3',
  caller: 'doc-qa',
  queued_ms: 200, // actual time spent in queue
  slot: 2, // which slot was assigned
  timestamp: '2025-01-15T10:23:45.200Z'
}

// 3. When a streaming token arrives for Meeting Summariser (continuous)
// NOTE: this is NOT a scheduler event – it's yielded directly via AsyncGenerator
{
  type: 'stream:token',
  data: { text: ' transcription fragment' },
  meta: {
    compute: 'ane',
    tier: 0,
    sequence: 47, // token sequence number
  }
}

// 4. When Meeting Summariser's stream ends
{
  type: 'stream:done',
  meta: {
    compute: 'ane',
    tier: 0,
    total_tokens: 312,
    duration_ms: 15400,
    latency_first_token_ms: 42,
  }
}

// 5. When Doc Q&A's inference completes
{
  status: 'ok',
  data: { text: 'नमस्ते, आप कैसे हैं?', source_lang: 'en', target_lang: 'hi' },
  meta: {

```

```

    compute: 'ane',
    tier: 0,
    latency_ms: 48,
    queued_ms: 200,           // ← queue wait visible in response meta
    request_id: 'req_a7f3',
  }
}

```

Scheduler internal state during contention

```

{
  "state": "saturated",
  "concurrency": {
    "max": 4,
    "effective_max": 4,
    "in_flight": 4,
    "per_caller": {
      "meeting-sum": { "in_flight": 2, "queued": 0, "max": 2, "streaming": 2 },
      "doc-qa": { "in_flight": 0, "queued": 1, "max": 2, "streaming": 0 }
    }
  },
  "queues": {
    "critical": { "length": 0, "virtual_time": 14.25 },
    "normal": { "length": 1, "virtual_time": 13.0 },
    "background": { "length": 0, "virtual_time": 12.0 }
  },
  "starvation": {
    "consecutive_high_priority": 0,
    "oldest_queued_age_ms": 0,
    "promotions_total": 0
  },
  "active_streams": [
    { "request_id": "req_8b1c", "caller": "meeting-sum", "type": "asr",
      "tokens_delivered": 312, "duration_ms": 15400 },
    { "request_id": "req_9d2e", "caller": "meeting-sum", "type": "tts",
      "chunks_delivered": 47, "duration_ms": 12300 }
  ]
}

```

5. Queue Visibility Rules

What MFEs can see

MFEs do **not** have direct access to the scheduler's internal state. They receive information through two channels:

1. **Event bus** — scheduler emits events that any MFE can subscribe to
2. **Response meta** — `queued_ms` field in the response tells the MFE how long its request waited

Visibility matrix

Information	Meeting Summariser	Document Q&A	Shell (DevTools)
Own stream tokens	Yes (AsyncGenerator)	N/A	Yes (event bus)
Own queue position	N/A (not queued)	Yes (scheduler:queued event)	Yes
Other MFE's queue position	No	No	Yes
Other MFE's streaming status	No	No	Yes
Total slots in use	No	No	Yes
Why request was queued	No	No (just "queued")	Yes (full state dump)
Queue wait time after completion	N/A	Yes (response.meta.queued_ms)	Yes

Design rationale: limited visibility

MFEs see only their **own** requests and results. They do not see other MFEs' state because:

1. **Isolation:** MFEs are developed by different teams. Exposing cross-MFE state creates coupling — if Doc Q&A's UI changes based on Meeting Summariser's stream count, both teams must coordinate.
2. **Security:** an MFE compromise should not leak information about other MFEs' workloads or usage patterns.
3. **Simplicity:** MFEs don't need to understand the scheduler's internal state to render correct UI. `queued_ms` and the event bus are sufficient.

Queue position event

The `scheduler:queued` event includes a `position` field. This is the request's position in the effective dispatch order (considering WFQ weights and per-MFE fairness), not its raw position in the FIFO lane.

```
// Position calculation
function effectivePosition(entry: QueueEntry): number {
  let position = 0;
  for (const lane of [queues.critical, queues.normal, queues.background]) {
    for (const queued of lane) {
      if (wouldDispatchBefore(queued, entry)) position++;
      if (queued === entry) return position + 1;
    }
  }
  return position + 1;
}
```

If Meeting Summariser submits a P0 request while Doc Q&A is at position 1, Doc Q&A moves to position 2 — the scheduler emits a `scheduler:position_changed` event:

```
{ type: 'scheduler:position_changed', request_id: 'req_a7f3', caller: 'doc-qa',  
old_position: 1, new_position: 2 }
```

6. Loading/Progress Semantics

Doc Q&A loading states

The MFE should display loading states based on scheduler events:

```
type LoadingState =  
  | { phase: 'submitting' } // inference() called,  
not yet acknowledged  
  | { phase: 'queued'; position: number; estimate_ms?: number } //  
scheduler:queued received  
  | { phase: 'processing' } // scheduler:dispatched  
received  
  | { phase: 'complete'; data: unknown; queued_ms: number } // response  
received  
  | { phase: 'error'; code: string; message: string }; // error received  
  
function DocQALoadingIndicator({ state }: { state: LoadingState }) {  
  switch (state.phase) {  
    case 'submitting':  
      return <Spinner label="Submitting..." />;  
    case 'queued':  
      return <Spinner label={`Queued - position ${state.position}`} />;  
    case 'processing':  
      return <Spinner label="Processing..." />;  
    case 'complete':  
      return null; // show result  
    case 'error':  
      return <ErrorBanner code={state.code} message={state.message} />;  
  }  
}
```

Meeting Summariser streaming states

The streaming MFE has a simpler model — it's either receiving tokens or it's not:

```
type StreamingState =  
  | { phase: 'connecting' } // stream() called, SSE not yet open  
  | { phase: 'streaming'; tokens: number } // tokens arriving  
  | { phase: 'done'; total_tokens: number; duration_ms: number }  
  | { phase: 'error'; code: string };  
  
function TranscriptionIndicator({ state }: { state: StreamingState }) {  
  switch (state.phase) {  
    case 'connecting':  
      return <Pulse label="Connecting to transcription..." />;  
  }  
}
```

```

    case 'streaming':
      return <LiveBadge label={`Live - ${state.tokens} tokens`} />;
    case 'done':
      return <Badge label="Transcription complete" />;
    case 'error':
      return <ErrorBanner code={state.code} />;
  }
}

```

Progress estimation

The `estimated_wait_ms` field in `scheduler:queued` is a best-effort estimate based on current throughput:

```

function estimateWait(position: number): number {
  const avgInferenceMs = recentResponseTimes.average(); // rolling window of last
  20 responses
  const slotsAvailable = maxConcurrent - totalInFlight;

  if (slotsAvailable > 0) return 0; // will dispatch on next dequeue

  // Estimate: position * average inference time / concurrent slots
  return Math.ceil(position * avgInferenceMs / maxConcurrent);
}

```

This estimate is deliberately conservative. It's better to say "500ms" and deliver in 200ms than to say "100ms" and deliver in 500ms.

7. Cancellation Behavior

Cancellation during contention

Three cancellation scenarios arise during contention:

Scenario A: Doc Q&A cancels while queued

The user navigates away from Doc Q&A while the query is still queued (not yet dispatched).

```

// Doc Q&A unmounts → cleanup fires
useEffect(() => {
  return () => scheduler.cancelByCaller('doc-qa');
}, []);

```

Result:

- Doc Q&A's queue entry is **removed** from the P1 lane
- **No slot was consumed** — nothing to free
- Meeting Summariser: **unaffected**
- Scheduler: emits `scheduler:cancelled { request_id, caller: 'doc-qa', was_queued: true }`

Scenario B: Doc Q&A cancels while dispatched (in-flight)

The user navigates away after the request was dispatched but before the response arrived.

Result:

- AbortController fires → HTTP connection closed → TCP RST to agent
- Agent drops the request from its internal queue
- Slot is freed
- Dequeue loop fires → if anything is queued, it dispatches
- Meeting Summariser: **unaffected**

Scenario C: Meeting Summariser cancels streams (user stops recording)

The user clicks "Stop Recording" in Meeting Summariser, which unmounts the component.

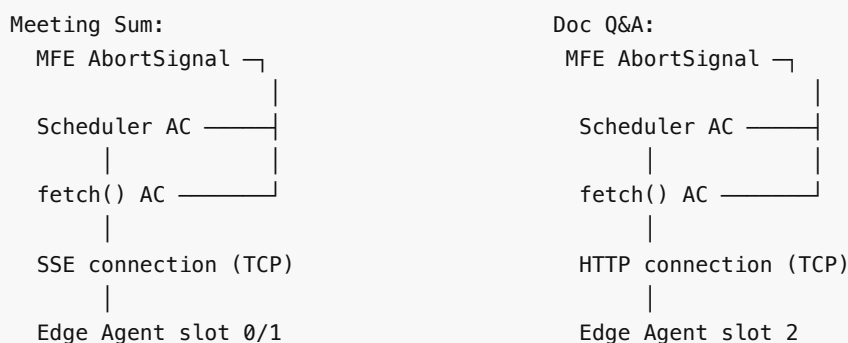
```
// Meeting Summariser unmounts
scheduler.cancelByCaller('meeting-sum');
```

Result:

- Both SSE streams are aborted → TCP RST sent for each
- 2 slots freed immediately
- Per-MFE quota recalculated: only 1 active MFE → doc-qa max = 4
- Dequeue loop fires → Doc Q&A's queued request dispatches immediately
- Doc Q&A: was at position 1, now dispatched with ~50ms queue time

This is the **fastest recovery path** — stream cancellation is instantaneous because AbortController signals are synchronous.

Cancellation and the AbortController chain



Each MFE's cancellation is independent. Cancelling Meeting Summariser's streams does not affect Doc Q&A's in-flight request, and vice versa.

8. Retry Semantics

When does the MFE retry?

The scheduler does **not** auto-retry on behalf of MFEs. Retry decisions belong to the MFE because only the MFE knows:

- Whether the user is still waiting (navigated away? tab backgrounded?)

- Whether a retry makes UX sense (real-time transcription can't retry — the audio moment is gone)
- Whether to modify the request (different model, lower quality, shorter input)

MFE retry contract

```
type RetryableResult = {
  status: 'error';
  code: 'timeout' | 'queue_full' | 'disconnected';
  retrying: false;
  message: string;
};
```

The `retrying: false` field tells the MFE: "the scheduler is not going to retry this for you." The MFE must decide.

Retry guidance by error code

Error code	Retry recommended?	MFE action	Delay
timeout	Yes (user-triggered)	Show "Timed out — Retry?" button	None — user decides
queue_full	Yes (after short wait)	Show "System busy — try again in a moment"	2-5s (exponential backoff)
disconnected	Auto-retry when reconnected	Subscribe to <code>connection:restored</code> event, then re-submit	Automatic
cancelled	No — MFE initiated it	No action needed	N/A
preempted	Automatic — scheduler re-queues	Invisible to MFE	N/A

Auto-retry on reconnect pattern

For the `disconnected` case, MFEs can use this pattern to auto-retry:

```
async function resilientInference(request: InferenceRequest):
Promise<InferenceResult> {
  const result = await inference(request);

  if (result.status === 'error' && result.code === 'disconnected') {
    // Wait for agent to come back
    await new Promise<void>((resolve) => {
      const unsub = events.on('connection:restored', () => {
        unsub();
        resolve();
      });
    });

    // Retry once after reconnection
    return inference(request);
  }
}
```

```
    }  
  
    return result;  
}
```

What about streaming retries?

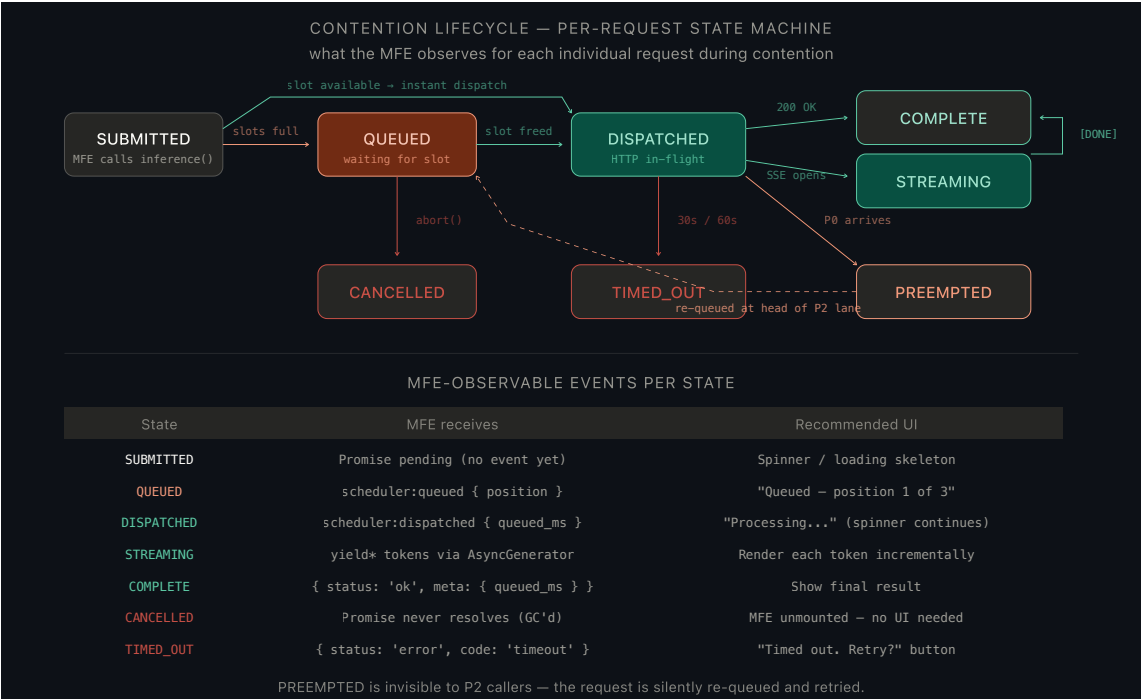
Streaming requests (SSE) cannot be resumed mid-stream after a disconnect. The entire stream must be re-initiated. Meeting Summariser should:

1. Detect stream interruption (AsyncGenerator throws or yields an error)
2. Buffer the partial transcription
3. Re-initiate `stream()` with fresh audio input
4. Append new tokens to the buffered transcription

```
async function* resilientStream(request: InferenceRequest):  
    AsyncGenerator<StreamChunk> {  
    try {  
        yield* stream(request);  
    } catch (error) {  
        if (error.code === 'stream_interrupted') {  
            // Wait for reconnect  
            await waitForReconnect();  
            // Re-initiate (caller must provide fresh audio input)  
            throw new StreamInterruptedError('Stream interrupted – re-initiate with  
current audio');  
        }  
        throw error;  
    }  
}
```

9. Process Lifecycle

The per-request lifecycle during contention has 7 states. This is the MFE-observable lifecycle — not the scheduler's internal state machine (covered in Part B §2).



States

State	Entry	Exit	MFE observes
SUBMITTED	MFE calls <code>inference()</code> / <code>stream()</code>	Scheduler acknowledges	Promise pending, no event yet
QUEUED	Scheduler enqueues (no slot available)	Slot becomes available	<code>scheduler:queued</code> event with position
DISPATCHED	Scheduler sends HTTP/SSE to agent	Response arrives, timeout, or abort	<code>scheduler:dispatched</code> event
STREAMING	SSE connection opens, first token arrives	[DONE] or error	Tokens via AsyncGenerator yield
COMPLETE	Full response received or [DONE]	Terminal	Promise resolves with <code>{ status: 'ok' }</code>
CANCELLED	MFE aborts or unmounts	Terminal	Promise never resolves (GC'd with MFE)
TIMED_OUT	30s (or 60s for P2) elapsed since SUBMITTED	Terminal	Promise resolves with <code>{ status: 'error', code: 'timeout' }</code>

State transitions

SUBMITTED —slot available→ DISPATCHED (instant dispatch, no queueing)
SUBMITTED —slots full→ QUEUED

```

QUEUED —slot freed→ DISPATCHED
QUEUED —abort()→ CANCELLED
QUEUED —30s elapsed→ TIMED_OUT
DISPATCHED —200 OK→ COMPLETE
DISPATCHED —SSE opens→ STREAMING
DISPATCHED —30s elapsed→ TIMED_OUT
DISPATCHED —P0 preempts (P2 only)→ QUEUED (re-queued at head)
STREAMING —[DONE]→ COMPLETE
STREAMING —abort()→ CANCELLED
STREAMING —30s elapsed→ TIMED_OUT

```

Implementation — per-request state tracking

```

class RequestTracker {
  private state: 'submitted' | 'queued' | 'dispatched' | 'streaming' | 'complete'
  | 'cancelled' | 'timed_out';

  constructor(
    readonly id: string,
    readonly caller: string,
    readonly priority: Priority,
    readonly submittedAt: number,
  ) {
    this.state = 'submitted';
  }

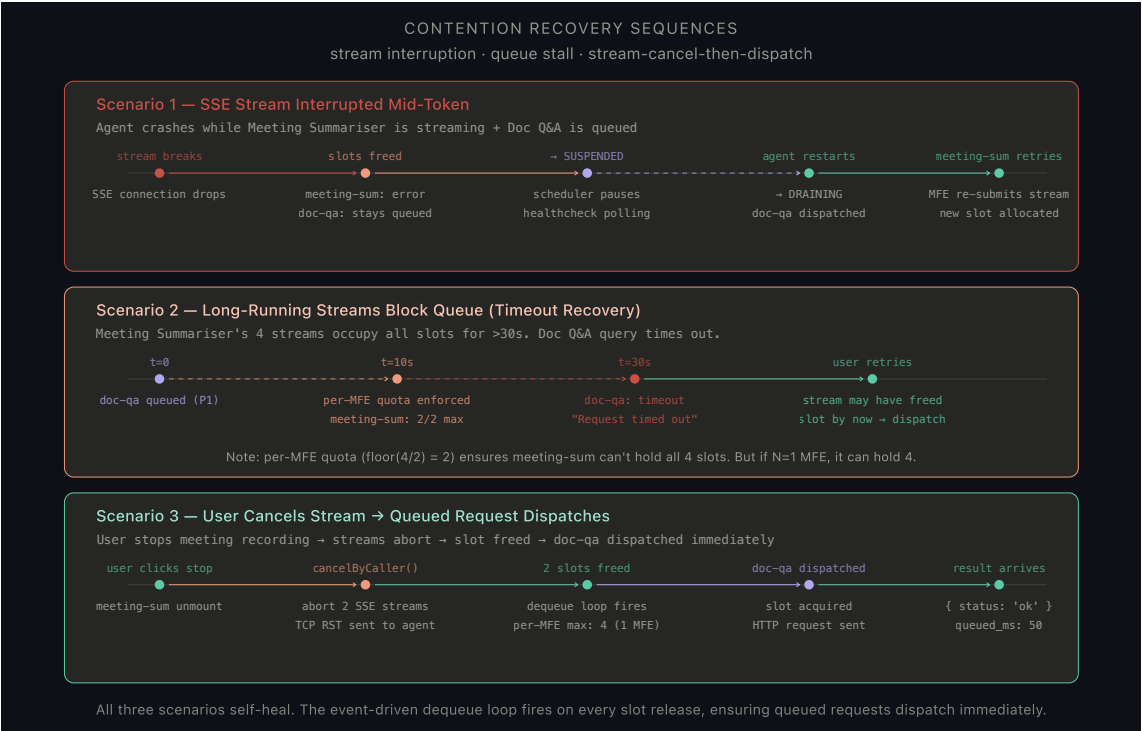
  transition(to: typeof this.state): void {
    const valid: Record<string, string[]> = {
      submitted: ['queued', 'dispatched'],
      queued: ['dispatched', 'cancelled', 'timed_out'],
      dispatched: ['streaming', 'complete', 'timed_out', 'queued'],
      streaming: ['complete', 'cancelled', 'timed_out'],
    };

    if (!valid[this.state]?.includes(to)) {
      throw new Error(`Invalid transition: ${this.state} → ${to}`);
    }
    this.state = to;
  }
}

```

10. Recovery Sequences

Three failure scenarios specific to streaming contention, with recovery flows.



Scenario 1 — SSE stream interrupted mid-token (agent crash)

Time	Event	Meeting Summariser	Document Q&A
t=0	Agent crashes, SSE connections drop	AsyncGenerator throws stream_interrupted	If queued: stays queued. If in-flight: gets disconnected error
t=0+	Scheduler transitions to SUSPENDED	Shows "Connection lost" banner	Shows "AI offline" or stays in queue
t=0-2s	Health-check detects agent is down	Buffers partial transcription	Waiting
t=Xs	Agent restarts, health-check passes		Queued request dispatches via DRAINING
t=Xs+	Scheduler transitions to DRAINING	Re-initiates stream (MFE decision)	Gets result, shows translation

Key behavior: Doc Q&A's queued request **survives** the agent restart if it hasn't timed out. The scheduler keeps the queue entry alive and dispatches it when the agent comes back.

Scenario 2 — Long-running streams block queue (timeout recovery)

If Meeting Summariser's streams run for > 30 seconds and no other slots free, Doc Q&A's request will timeout.

Time	Event	Meeting Summariser	Document Q&A
t=0	Doc Q&A query queued	Streaming normally	Shows "Queued — position 1"

t=10s	Age-based promotion check: not yet	Streaming normally	Still queued (P1)
t=20s	Age promotion: P1→P0 (20s threshold)	Streaming normally	Promoted to P0 — now at WFQ head
t=30s	Timeout fires	Streaming normally	Gets { error: 'timeout' }
t=30s+	User clicks "Retry"	Stream may have ended by now	New request submitted, may dispatch immediately

Key behavior: Age-based promotion moved Doc Q&A's request to P0 at t=20s, giving it the highest WFQ priority. But if no slot frees (all 4 are long-running streams), promotion doesn't help — the request still times out at 30s. The timeout is the hard upper bound.

Why per-MFE quota helps: With $\text{floor}(4/2) = 2$, Meeting Summariser can only hold 2 slots. The other 2 slots are available for Doc Q&A or other MFEs. In the worst case (all 4 slots held by Meeting Summariser because it was the only MFE when it started), the quota is recalculated when Doc Q&A appears — but existing in-flight requests are not preempted by quota changes. They're only bounded by timeout.

Scenario 3 — User cancels stream, queued request dispatches

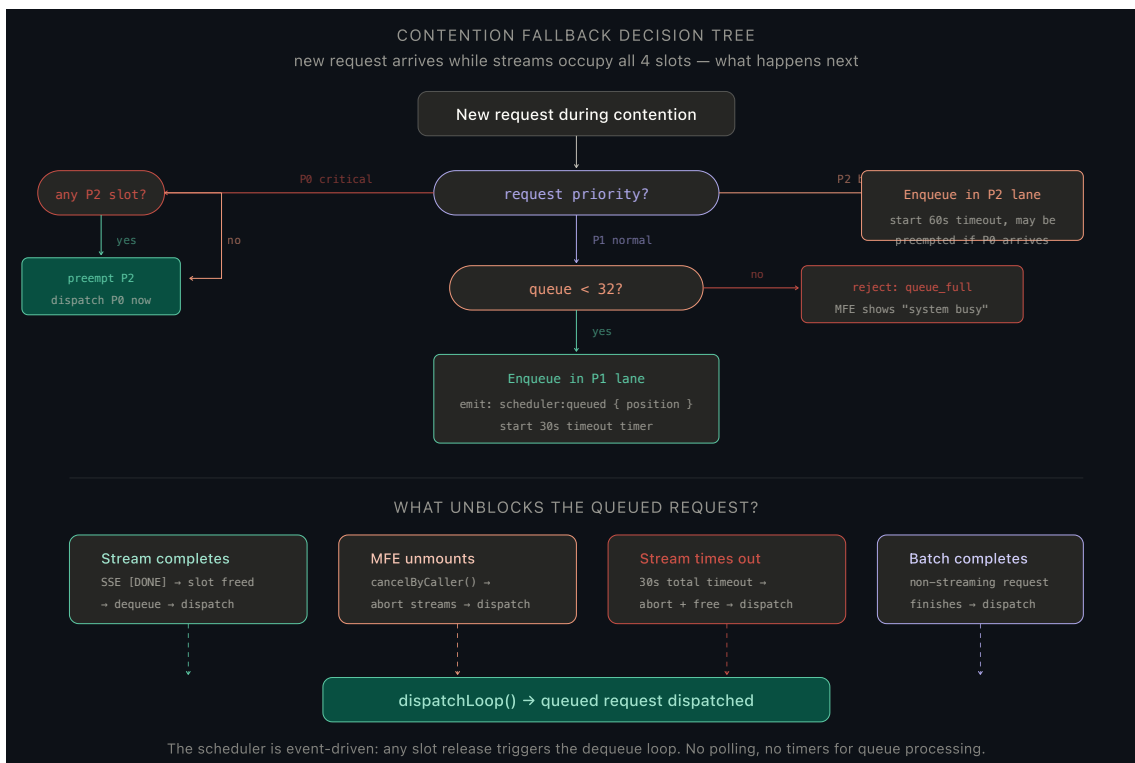
This is the happy recovery path. When Meeting Summariser stops, slots free instantly.

Time	Event	Meeting Summariser	Document Q&A
t=0	Doc Q&A query queued (4/4 full)	Streaming (2 slots)	Shows "Queued"
t=Xs	User clicks "Stop Recording"	MFE unmounts, <code>cancelByCaller('meeting-sum')</code> fires	
t=Xs+0ms	2 SSE streams aborted (TCP RST)	Cleaned up	Dequeue loop fires
t=Xs+1ms	Slots freed, quota recalculated		Dispatched (was position 1)
t=Xs+50ms			Result arrives, shows translation

Key behavior: The total queue time for Doc Q&A is however long the user took to stop the recording. The dispatch is near-instant once slots free.

11. Fallback Decision Tree

When a new request arrives during contention, the scheduler walks this decision tree.



Decision flow specific to contention

New request arrives, 4/4 slots occupied by streams + batch

- ├ Is it P0 (critical)?
 - ├ Any P2 (background) in-flight? → preempt P2, dispatch P0
 - └ All in-flight are P0/P1? → enqueue in P0 lane (wait for slot)
- ├ Is it P1 (normal)? ← Doc Q&A's case
 - ├ Queue < 32? → enqueue in P1 lane, start 30s timeout
 - ├ Emit scheduler:queued { position }
 - ├ Wait for slot to free (stream ends, batch completes, timeout, cancel)
 - └ On slot free → dispatch (WFQ selects, per-MFE quota check)
 - └ Queue ≥ 32? → reject with { error: 'queue_full' }
- └ Is it P2 (background)?
 - ├ Enqueue in P2 lane, start 60s timeout
 - ├ May be preempted if P0 arrives later
 - └ Promotes to P1 after 10s, to P0 after 20s

Why P1 cannot preempt P0 streams

Preemption is limited to P0→P2 by design. If P1 could preempt P0:

- A user clicking "Translate" in Doc Q&A would interrupt live transcription
- The Meeting Summariser user sees their transcript cut off mid-sentence
- The aborted stream is wasted work — SSE can't be resumed mid-stream

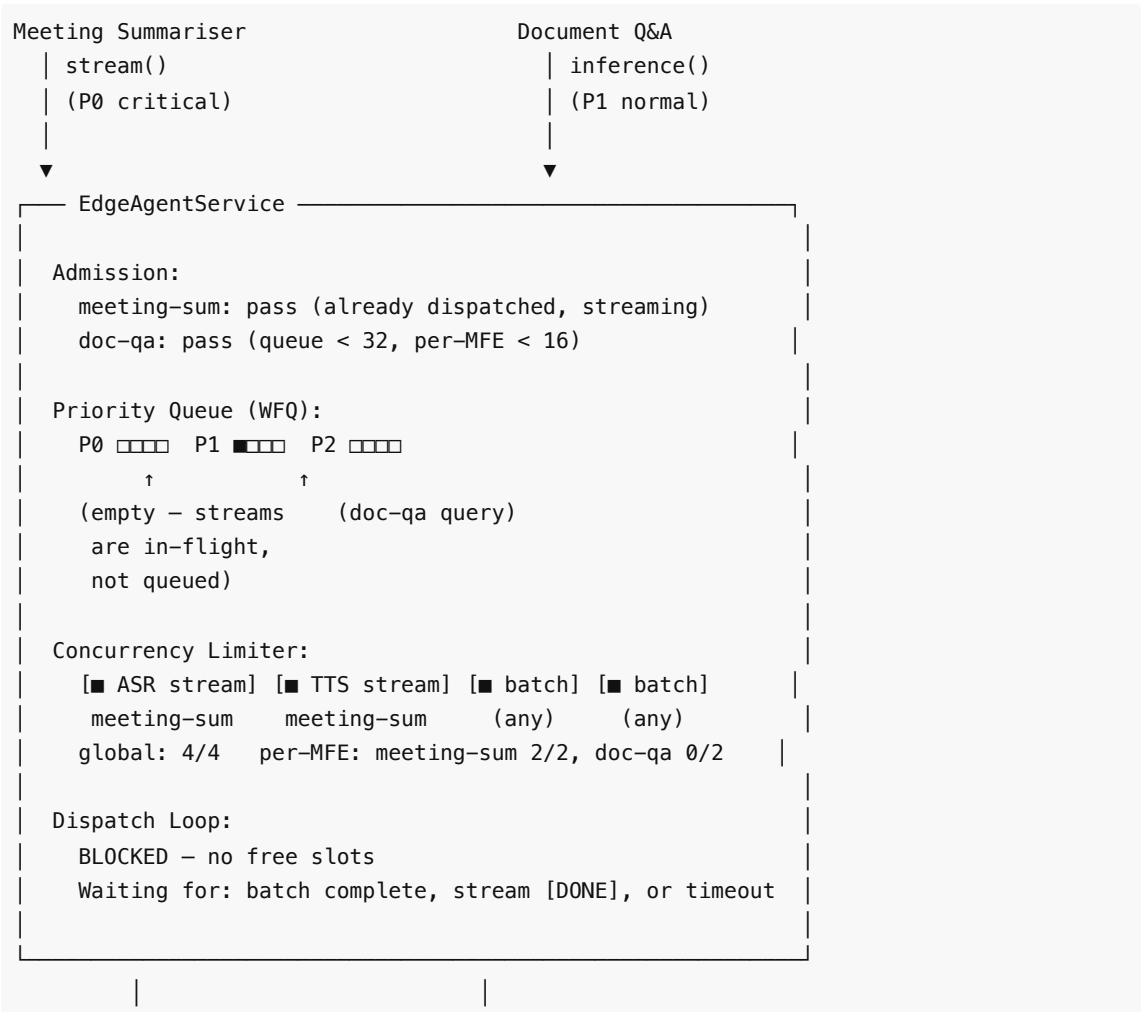
- The UX cost (interrupted transcription) far exceeds the UX benefit (faster translation)

Fallback → MFE UI mapping for contention

Outcome	Doc Q&A sees	Meeting Summariser sees
Queued briefly (< 1s)	Spinner → result (barely noticeable delay)	Nothing — uninterrupted
Queued longer (1-10s)	"Queued — position 1" → "Processing..." → result	Nothing
Queued, promoted (10-20s)	Position updates as promotion changes WFQ priority	Nothing
Timeout (30s)	"Request timed out. Retry?"	Nothing
Queue full (32/32)	"System busy — please try again"	Nothing — its streams are fine

12. Scheduler Architecture — Contention View

The architecture during contention, showing how data flows through each layer:



30s timeout includes queue wait	Single timer from submission to completion	Separate queue timeout + inference timeout	Users don't distinguish "waited 25s in queue + 5s inference" from "waited 5s in queue + 25s inference." Both feel like 30s. Single timeout is simpler and matches UX expectations.
Token forwarding (no buffering)	SSE events forwarded immediately to MFE	Buffer tokens, deliver in batches	Buffering adds latency to real-time transcription. For live ASR, every millisecond matters. The scheduler's job is routing, not transformation.
Event bus for queue updates	MFE subscribes to scheduler events	Poll scheduler state from MFE	Event-driven is more efficient and lower-latency. Polling would add unnecessary overhead and would need a polling interval decision.
estimated_wait_ms as best-effort	Conservative estimate based on rolling average	Guaranteed SLA or no estimate	Guarantees are impossible when stream duration is unknown. Best-effort with conservative bias (over-estimate) prevents user frustration from broken promises.

14. Performance & Reliability

Performance during contention

Metric	Value	Notes
Enqueue overhead (Doc Q&A)	~0.1ms	Create QueueEntry, insert in P1 lane, emit event
Token forwarding latency	< 0.01ms	AsyncGenerator yield — no buffering, no copy
Slot release → dequeue → dispatch	< 0.15ms	Event-driven, synchronous within microtask
Queue position calculation	~0.05ms	Iterate at most 32 entries
Total Doc Q&A queue time (typical)	50–500ms	Depends on when a batch slot frees
Total Doc Q&A queue time (worst case)	30s (timeout)	All 4 slots are long-running streams
Memory overhead per queued request	~0.5KB	QueueEntry + AbortController + timeout handle

Throughput under streaming contention

Scenario	Meeting Sum slots	Doc Q&A slots	Queue depth	Doc Q&A wait
----------	-------------------	---------------	-------------	--------------

2 streams + 2 short batches	2	0 → 1 (on batch complete)	1	50–200ms
2 streams + 2 long batches	2	0 → 1 (on batch complete)	1	200–2000ms
4 streams (all meeting-sum, N=1 MFE)	4	0 (blocked until stream ends)	1+	Up to 30s (timeout)
2 streams, doc-qa burst (5 queries)	2	2 (per-MFE max)	3	200ms–5s (WFQ rotation)

Reliability guarantees

Concern	Mitigation	Tested by scenario
Stream interruption	Streams fail independently. Other MFE unaffected. Queued requests survive if timeout hasn't elapsed.	Scenario 1
Queue stall from long streams	30s timeout is the hard upper bound. Age-based promotion escalates priority. Per-MFE quota limits monopolization.	Scenario 2
Slot leak from stream abort	AbortController → TCP RST → agent frees slot. Sweep timer (60s) as safety net.	Scenario 3
Fairness under contention	Per-MFE quota: floor(4/N). Doc Q&A can't be permanently starved if it has quota headroom.	All scenarios
WFQ correctness during contention	Streaming slots are counted in per-MFE in_flight, not in the queue. WFQ operates only on queued entries.	All scenarios
Event ordering	scheduler:queued always fires before scheduler:dispatched. Promise resolution always fires after scheduler:dispatched. SSE tokens are in-order (HTTP/2 stream ordering).	Timeline (§3)

Edge cases

Edge case	Behavior	Resolution
Doc Q&A submits while queue already has 31 entries	Enqueued at position 32 (queue not full)	If 32nd entry: next request gets queue_full
Meeting Summariser opens a 3rd stream (P0)	Would exceed per-MFE quota (2/2)	Queued in P0 lane — still has WFQ priority, dispatched when slot frees
Both MFEs submit P0 at the same time	WFQ virtual time breaks the tie	The one with lower virtual finish time dispatches first
Agent returns 429 while streams are active	Scheduler enters BACKPRESSURE	Streams continue (already dispatched). New dispatches paused.

Doc Q&A's query promoted to P0, but P0 stream fills all slots	Promotion changes WFQ priority but doesn't create slots	Still waits for a slot to free; timeout at 30s
---	---	--

Summary

Question	Answer
What Meeting Summariser receives	Uninterrupted SSE token stream. No latency change, no contention notifications, no awareness of Doc Q&A's queue state. Tokens forwarded immediately (< 0.01ms overhead).
What Document Q&A receives	<code>scheduler:queued</code> event with position → <code>scheduler:dispatched</code> event → { status: 'ok', meta: { queued_ms } }. Typical wait: 50–500ms. Hard upper bound: 30s timeout.
When updates are delivered	<code>scheduler:queued</code> — synchronous with <code>inference()</code> call. <code>scheduler:dispatched</code> — when slot is acquired. Tokens — real-time, never buffered. Promise — after full response.
Event/state formats	Typed TypeScript events with <code>request_id</code> , <code>caller</code> , <code>priority</code> , <code>position</code> , <code>queued_ms</code> . All events on the <code>EdgeAgentService</code> event bus.
Queue visibility rules	MFEs see only their own requests. No cross-MFE visibility. Shell DevTools can see full state. <code>queued_ms</code> in response meta reveals wait time.
Loading/progress semantics	4-phase model: submitting → queued (with position) → processing → complete/error. Streaming: connecting → streaming (with token count) → done/error.
Cancellation behavior	<code>AbortController</code> chain propagates to agent. Cancel while queued: entry removed, no slot consumed. Cancel while in-flight: slot freed, <code>dequeue</code> fires. MFEs cancel independently.
Retry semantics	Scheduler does not auto-retry. MFE decides based on error code. <code>timeout</code> : user-triggered retry. <code>disconnected</code> : auto-retry on reconnect. Streaming cannot resume — must re-initiate.